Computer Networks · Project Documentation

Video Conferencing Application (Mini-Meet)

Project number	16
Project title	Video Conferencing Application (Mini-Meet) Group B: Real-World Application Projects
Team members	Suvansh Choudhary (25BAI0252) Priyam Sharma (25BAI0159) Avika Agarwal (25BBS0057)
Source code	https://github.com/TowVansh/mini-meet

Expected outcomes and status

No.	Expected outcome	Status
1	Working multi-party call with audio and video	**Done**
2	Custom signalling server and documented signalling protocol	**Done**
3	NAT traversal demonstration across separate networks (STUN)	**Done**
4	Call-quality measurements under packet loss, jitter and bandwidth limits	**Done**
5	Architecture document covering media path vs. signalling path	**Done**

Outcome 1: Working multi-party call with audio and video **Done**

Each client captures from the webcam and microphone, encodes VP8 video and Opus audio, and sends RTP packets over UDP directly to every other participant (full mesh, up to 4 people). Each video frame is split into packets that fit the MTU, and each packet is copied to every connected peer.

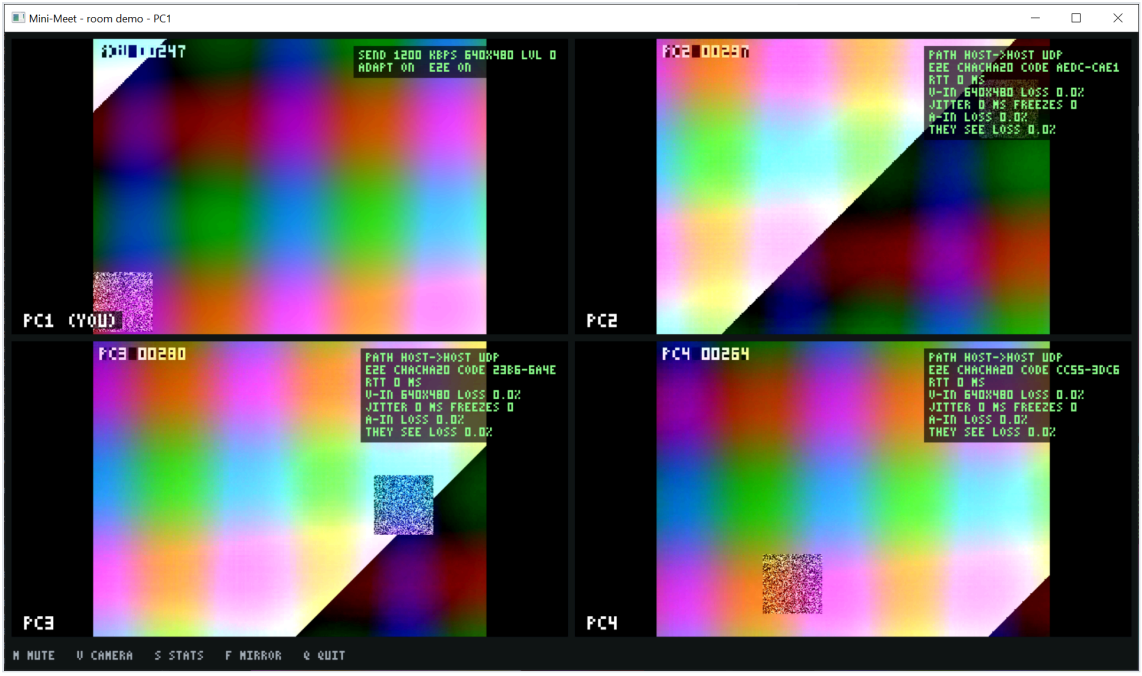
Code

src/client/media_in.c

```
static void fanout(const uint8_t *pkt, size_t len, int video) {
    int i;
    for (i = 0; i < MAX_PEERS; i++) {
        peer_t *p = &g.peers[i];
        if (!media_ready(p)) continue;
        send_rtp_to(p, pkt, len);
        if (video) p->tx_bytes_v += len;
        else p->tx_bytes_a += len;
    }
}

void send_video_frame(const uint8_t *data, size_t len, int key, uint32_t ts) {
    const size_t chunk = RTP_MAX_PAYLOAD - 1;
    size_t off = 0;
    SDL_LockMutex(g.lock);
    while (off < len) {
        uint8_t pkt[PKT_MAX];
        size_t n = len - off < chunk ? len - off : chunk, plen;
        rtp_hdr_t h;
        h.marker = off + n >= len;          /* marker = last packet of this frame */
        h.pt = RTP_PT_VP8;
        h.seq = g.vseq++;
        h.ts = ts;
        h.ssrc = g.vssrc;
        plen = rtp_write_header(pkt, &h);
        pkt[plen++] = (uint8_t)((off == 0 ? VP_START : 0) | (key ? VP_KEY : 0));
        memcpy(pkt + plen, data + off, n);
        plen += n;
        {   /* keep a copy for NACK retransmission */
            int slot = h.seq % TX_HISTORY;
            g.txhist[slot].used = 1;
            g.txhist[slot].seq = h.seq;
            g.txhist[slot].len = (uint16_t)plen;
            g.txhist[slot].sent_us = now_us();
            memcpy(g.txhist[slot].data, pkt, plen);
        }
        fanout(pkt, plen, 1);
        g.vpackets++;
        g.voctets += (uint32_t)n;
        off += n;
    }
    g.last_vts = ts;
    SDL_UnlockMutex(g.lock);
}
```

Implementation



Four clients in one call, seen from PC1, using the built-in test video and tone. Each remote tile shows its live statistics.

All 12 directed video streams of a 4-person call arrived at 640×480 and 20 fps, about 1190–1220 kbps each. The webcam and microphone call is shown live in the demo.

Outcome 2: Custom signalling server and documented signalling protocol Done

`mm-server` is a single-threaded `select()` server. It groups clients into rooms of up to 4 and relays network candidates and public keys inside a room. Every message is one line of text over TCP. The full protocol is documented in `docs/PROTOCOL.md`.

Client to server	Server to client
`JOIN <room> <name>`	`JOINED <id> <room> [id:name ...]`
`CAND <to> <type> <ip> <port>`	`PEER_JOINED <id> <name>`, `PEER_LEFT <id>`
`KEY <to> <64 hex>`	`CAND <from> ...`, `KEY <from> ...`
`LEAVE`, `PING`	`PONG`, `ERROR <code> <text>`

Code

src/server/mm_server.c

```
static void handle_join(client_t *c, mm_msg_t *m) {
    char line[MM_LINE_MAX];
    size_t len;
    int i;
    if (m->argc != 3) return send_error(c, "BAD_MESSAGE", "usage: JOIN <room> <name>");
    if (c->room[0]) return send_error(c, "ALREADY_JOINED", "already in a room");
    if (!mm_valid_name(m->argv[1]) || !mm_valid_name(m->argv[2]))
        return send_error(c, "BAD_NAME", "room and name must be 1-32 chars of A-Z a-z 0-9 _ -");
    if (room_count(m->argv[1]) >= MM_MAX_ROOM) return send_error(c, "ROOM_FULL", "room already has 4
participants");

    snprintf(c->name, sizeof c->name, "%s", m->argv[2]);
    snprintf(c->room, sizeof c->room, "%s", m->argv[1]);

    /* JOINED lists everyone already in the room; they get PEER_JOINED. */
    len = (size_t)snprintf(line, sizeof line, "JOINED %s %s", c->id, c->room);
    for (i = 0; i < nclients; i++) {
        client_t *o = &clients[i];
        if (o == c || strcmp(o->room, c->room) != 0) continue;
        len += (size_t)snprintf(line + len, sizeof line - len, " %s:%s", o->id, o->name);
        send_line(o, "PEER_JOINED %s %s", c->id, c->name);
    }
    send_line(c, "%s", line);
    {
        char a[32];
        log_msg("room", "join room=%s id=%s name=%s from=%s (%d/%d)", c->room, c->id, c->name,
                ep_str(&c->addr, a, sizeof a), room_count(c->room), MM_MAX_ROOM);
    }
}

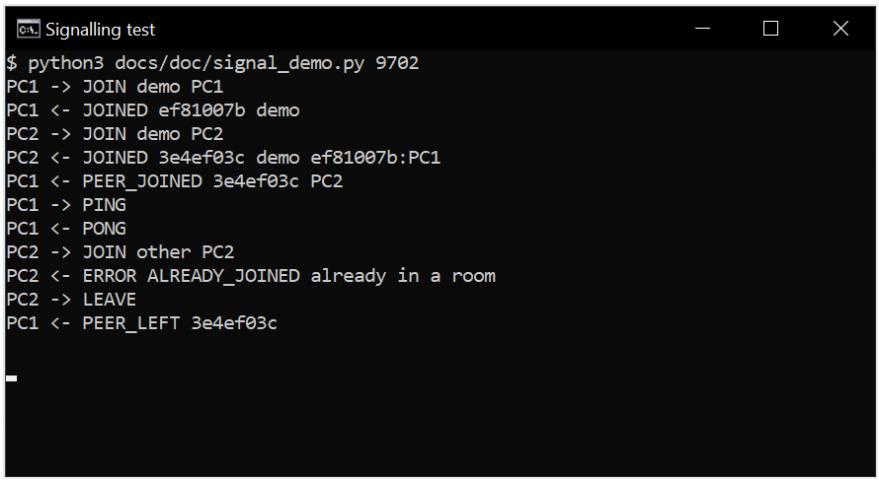
static void handle_cand(client_t *c, mm_msg_t *m) {
    client_t *to;
    endpoint_t ep;
    if (m->argc != 5) return send_error(c, "BAD_MESSAGE", "usage: CAND <to> <type> <ip> <port>");
    if (!c->room[0]) return send_error(c, "NOT_JOINED", "join a room first");
    if (!mm_valid_cand_type(m->argv[2]) || ep_parse(m->argv[3], m->argv[4], &ep) != 0)
        return send_error(c, "BAD_MESSAGE", "bad candidate");
    to = find_in_room(c->room, m->argv[1]);
    if (!to) return send_error(c, "UNKNOWN_PEER", "target peer is not in your room");
    /* Relay with the sender's id in place of the target's. */
    send_line(to, "CAND %s %s %s %s", c->id, m->argv[2], m->argv[3], m->argv[4]);
}
```

src/common/proto.c

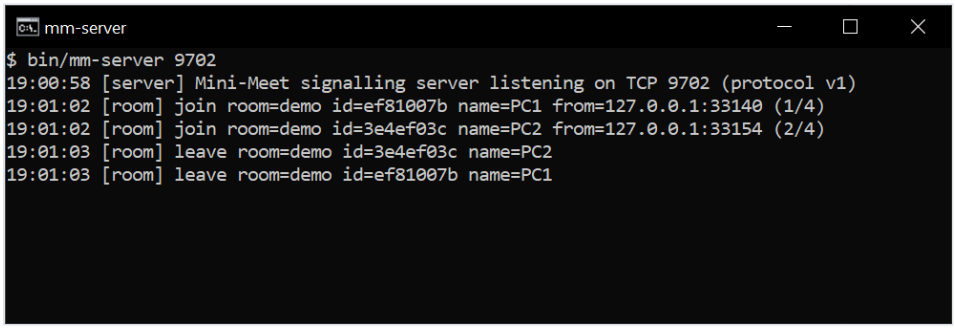
```
int mm_linebuf_pop(mm_linebuf_t *lb, char *out, size_t cap) {
    char *nl = memchr(lb->buf, '\n', lb->len);
    size_t line_len;
    if (!nl) return 0;
    line_len = (size_t)(nl - lb->buf);
```

```
    if (line_len >= cap) line_len = cap - 1;
    memcpy(out, lb->buf, line_len);
    out[line_len] = '\0';
    line_len = (size_t)(nl - lb->buf) + 1;
    memmove(lb->buf, lb->buf + line_len, lb->len - line_len);
    lb->len -= line_len;
    return 1;
}
```

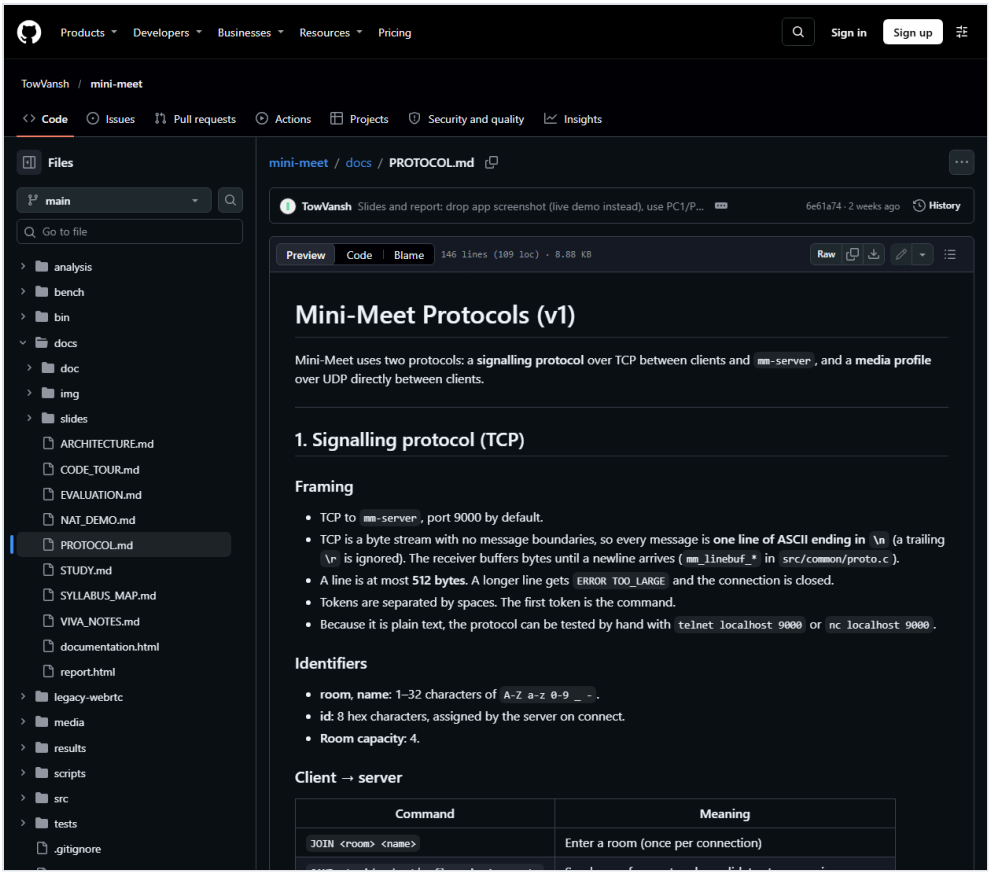
Implementation



Protocol exchange with the running server: two clients join, ping, a rejected second JOIN, and a leave.



Server log for the same session.



The protocol document, docs/PROTOCOL.md, in the repository.

Outcome 3: NAT traversal demonstration across separate networks (STUN) Done

Each client asks a STUN server for its public address (the server-reflexive, or srflx, candidate) and sends its candidates through signalling. Both peers then send STUN checks to each other at the same time, which opens a path through both NATs (UDP hole punching). The first candidate that answers carries the media.

Code

src/stun/stun.c

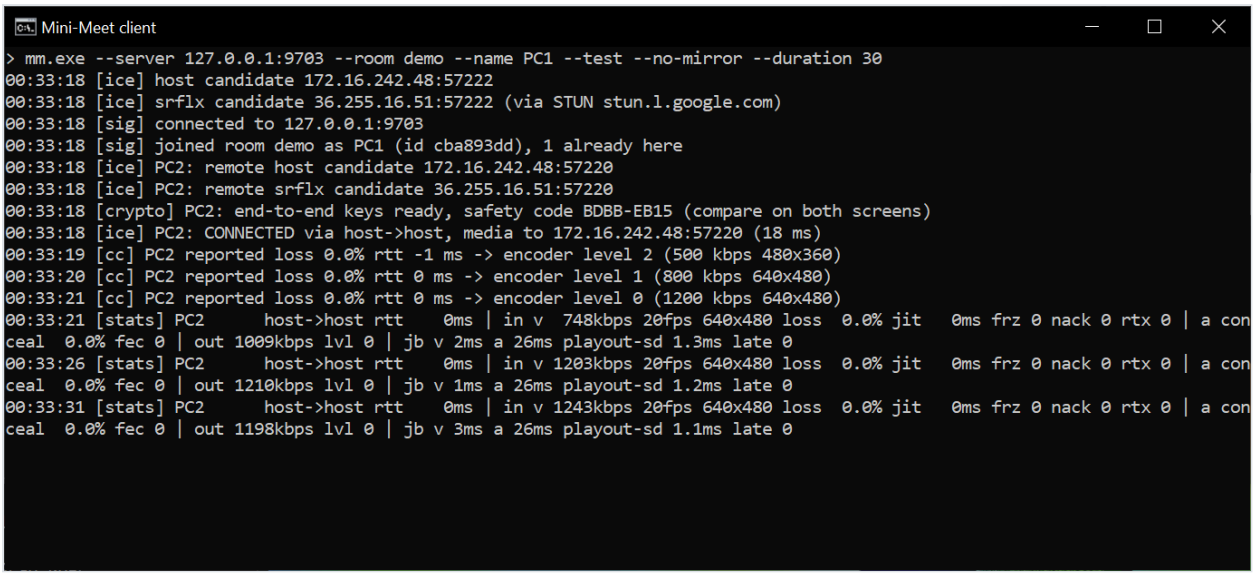
```
size_t stun_build_success(uint8_t *out, size_t cap, const uint8_t tid[12], const endpoint_t *mapped) {
    uint8_t v[8];
    uint32_t ip_host = ntohl(mapped->ip);
    size_t n;
    if (cap < STUN_HEADER_LEN) return 0;
    n = put_header(out, STUN_BINDING_SUCCESS, tid);
    /* XOR-MAPPED-ADDRESS: port XOR top 16 bits of cookie, address XOR cookie.
     * Obfuscating it stops NAT "helpers" that rewrite IPs inside payloads. */
    v[0] = 0;
    v[1] = 0x01; /* IPv4 */
    put_u16(v + 2, (uint16_t)(mapped->port ^ (STUN_MAGIC_COOKIE >> 16)));
    put_u32(v + 4, ip_host ^ STUN_MAGIC_COOKIE);
    n = put_attr(out, n, cap, STUN_ATTR_XOR_MAPPED_ADDRESS, v, 8);
    n = put_attr(out, n, cap, STUN_ATTR_SOFTWARE, "mini-meet", 9);
    finish(out, n);
    return n;
}
```

src/client/ice.c

```
static void send_check(peer_t *p, cand_t *c) {
    uint8_t buf[128];
    char user[2 * MM_ID_LEN + 2];
    snprintf(user, sizeof user, "%s:%s", p->id, g.self_id);
    udp_send(g.udp, &c->addr, buf, stun_build_request(buf, sizeof buf, c->tid, user));
}

void ice_tick(peer_t *p, uint64_t now) {
    int i;
    if (p->ice_state == ICE_CHECKING) {
        if (now - p->last_check_ms >= CHECK_INTERVAL_MS) {
            p->last_check_ms = now;
            for (i = 0; i < p->ncands; i++) send_check(p, &p->cands[i]);
        }
        if (now - p->ice_start_ms > ICE_TIMEOUT_MS) {
            p->ice_state = ICE_FAILED;
            log_msg("ice", "%s: no candidate pair worked after %d s. Both sides behind symmetric NAT? "
                    "That case needs a TURN relay.", p->name, ICE_TIMEOUT_MS / 1000);
        }
    } else if (p->ice_state == ICE_CONNECTED) {
        /* Keepalive: refresh NAT mappings (they expire after ~30 s of silence). */
        if (now - p->last_keepalive_ms >= KEEPALIVE_MS) {
            cand_t c;
            p->last_keepalive_ms = now;
            c.addr = p->sel;
            stun_new_tid(c.tid);
            send_check(p, &c);
        }
    }
}
```

Implementation



Windows client run live: STUN returns the public (srflx) address of this laptop. In this run both clients are on one laptop, so the direct host path connects first.



Saved client log of the cross-network test on 2026-10-08: this laptop on college Wi-Fi, the other (Friend) on a phone hotspot. Different private and public addresses; connected directly, srflx to srflx. The second command counts the path recorded in every sample of that call.

The test recorded 1404 samples (`results/nat_c__Laptop.csv`), all on `srflx->srflx`, with a median round-trip time of 85 ms. Signalling went through a TCP tunnel; media went directly between the laptops. Not covered: two symmetric NATs would need a TURN relay, which is not implemented. The client detects this case and reports it after 15 s.

Outcome 4: Call-quality measurements under packet loss, jitter and bandwidth limits **Done**

Two Linux network namespaces joined by a virtual link act as two hosts, with `tc netem` impairing each direction. Each of 14 profiles ran with rate control off and on (28 runs). Every client records bitrate, fps, loss, jitter, round-trip time, freezes and audio concealment each second. The full analysis is in `docs/EVALUATION.md`.

Condition	Rate control	fps	Loss %	Round-trip ms	Freezes	Audio concealed %
No impairment	on	20	0.0	0.7	0	0.0
Packet loss 5 %	off	20	4.9	0.3	0	0.2
Packet loss 20 %	off	19	19.8	0.2	23	4.0
Jitter ±60 ms	off	20	1.7	194	4	0.2
Bandwidth 1 Mbit/s	off	15	13.6	10513	0	34.1
Bandwidth 1 Mbit/s	on	20	0.0	2.8	0	2.7
Bandwidth 500 kbit/s	off	0	39.6	19414	2	49.0
Bandwidth 500 kbit/s	on	20	0.0	9.8	0	11.7

Code

scripts/impair.sh

```
profile_args() {
  case "$1" in
    baseline)    echo "" ;;
    loss1)       echo "loss 1%" ;;
    loss5)       echo "loss 5%" ;;
    loss10)      echo "loss 10%" ;;
    loss20)      echo "loss 20%" ;;
    burst5)      echo "loss gemodel 1% 30% 70% 0.1%" ;;   # bursty (Gilbert-Elliott)
    delay100)    echo "delay 100ms" ;;
    jitter30)    echo "delay 100ms 30ms distribution normal" ;;
    jitter60)    echo "delay 100ms 60ms distribution normal" ;;
    rate2m)      echo "rate 2mbit" ;;
    rate1m)      echo "rate 1mbit" ;;
    rate500k)    echo "rate 500kbit" ;;
    rate250k)    echo "rate 250kbit" ;;
    combo)       echo "delay 80ms 20ms distribution normal loss 3% rate 1mbit" ;;
    *)           return 1 ;;
  esac
}
```

src/client/rtp.c

```
void rx_stats_update(rx_stats_t *s, uint16_t seq, uint32_t rtp_ts, uint64_t arrival_us) {
    uint16_t delta = (uint16_t)(seq - s->max_seq);
    int64_t arrival, transit, d;

    /* Sequence tracking: a forward jump extends max_seq (counting wraps). */
    if (delta > 0 && delta < 3000) {
        if (seq < s->max_seq) s->cycles += 65536;
        s->max_seq = seq;
    } else if (delta > 65536 - 100 || delta == 0) {
        /* duplicate or reordered packet: counted as received below */
    } else {
        /* huge jump: the sender restarted; resync */
        s->base_seq = seq;
        s->max_seq = seq;
        s->cycles = 0;
        s->received = 0;
        s->expected_prior = s->received_prior = 0;
```

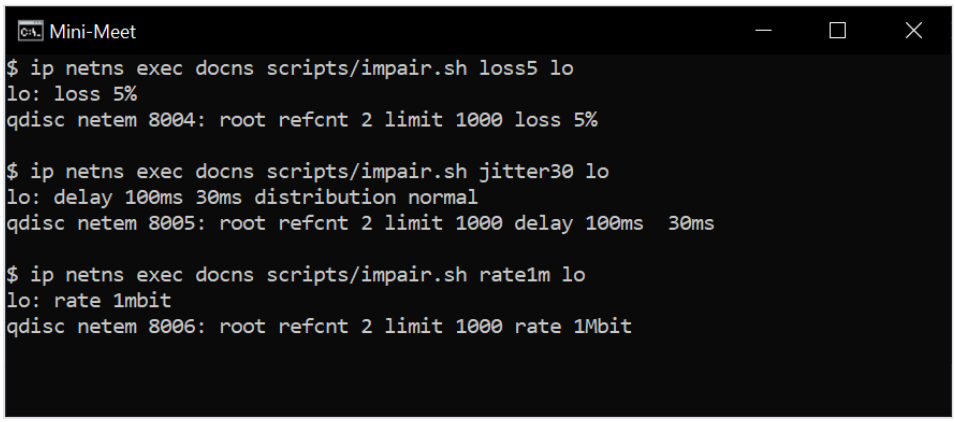
```
    }
    s->received++;

    /* Interarrival jitter, RFC 3550 6.4.1: J += (|D| - J) / 16, in RTP units. */
    arrival = (int64_t)(arrival_us * s->clock_rate / 1000000ull);
    transit = arrival - (int64_t)rtp_ts;
    if (s->have_transit) {
        d = transit - s->last_transit;
        if (d < 0) d = -d;
        s->jitter += ((double)d - s->jitter) / 16.0;
    }
    s->last_transit = transit;
    s->have_transit = 1;
}
```

src/client/cc.c

```
int cc_on_report(cc_state_t *cc, double loss, double rtt_ms) {
    int old = cc->level;
    double qdelay;
    if (rtt_ms >= 0) {
        /* Smooth out jitter in single samples (like TCP's SRTT, alpha = 1/4). */
        cc->srtt_ms = cc->srtt_ms < 0 ? rtt_ms : 0.75 * cc->srtt_ms + 0.25 * rtt_ms;
        if (cc->min_ever_ms < 0 || cc->srtt_ms < cc->min_ever_ms) cc->min_ever_ms = cc->srtt_ms;
        /* Window holds smoothed values too, so random jitter in single samples
         * does not drag the baseline down and look like queuing delay. */
        cc->rtt_hist[cc->hist_pos] = cc->srtt_ms;
        cc->hist_pos = (cc->hist_pos + 1) % CC_RTT_WINDOW;
        if (cc->nhist < CC_RTT_WINDOW) cc->nhist++;
    }
    qdelay = cc_queuing_delay(cc);
    if (loss > LOSS_BAD || qdelay > QDELAY_BAD) {
        cc->level += 2;
        if (cc->level >= CC_LEVELS) cc->level = CC_LEVELS - 1;
        cc->clean = 0;
        cc->startup = 0;
    } else if (loss < LOSS_OK && qdelay < QDELAY_OK) {
        /* Startup: climb one level per clean report until the first congestion
         * signal (fast ramp like TCP slow start), then the slower AIMD rule. */
        if (++cc->clean >= (cc->startup ? 1 : CLEAN_ROUNDS)) {
            if (cc->level > 0) cc->level--;
            cc->clean = 0;
        }
    } else {
        cc->clean = 0;
    }
    return cc->level != old;
}
```

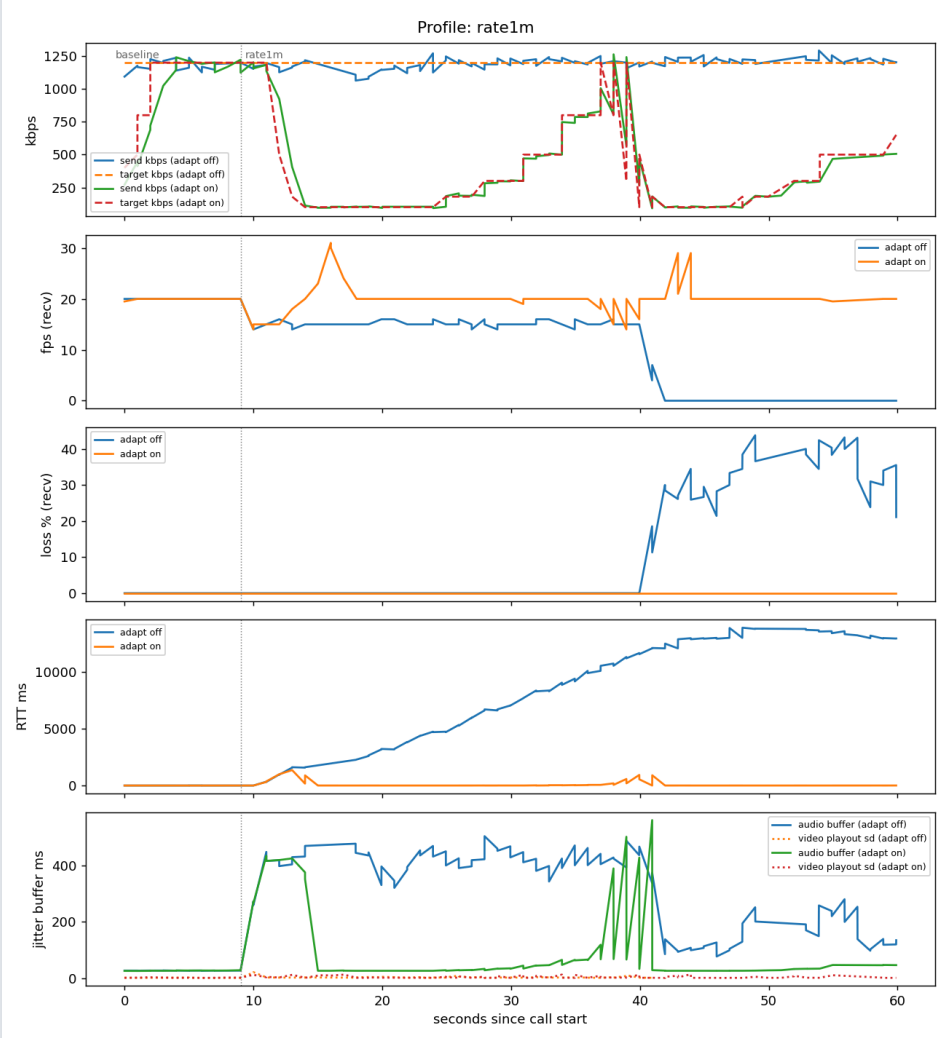
Implementation



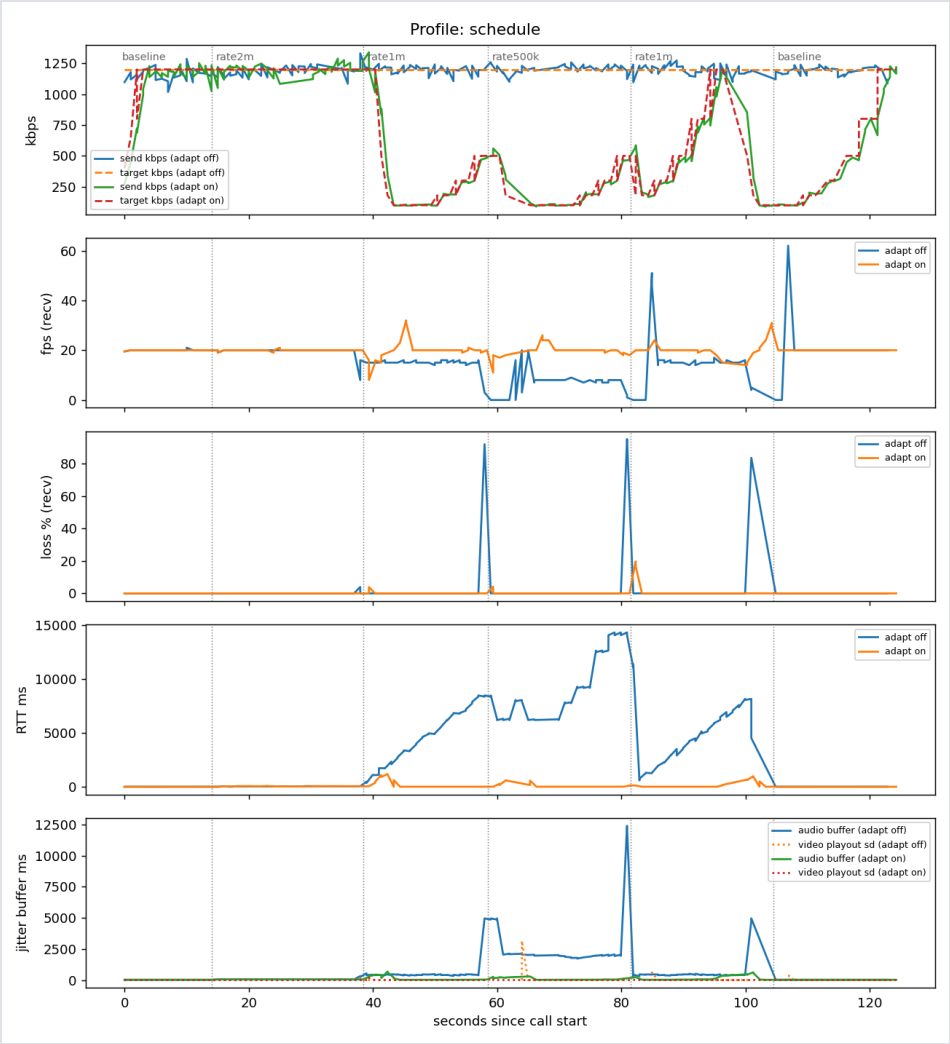
Impairment profiles applied with tc netem.

```
Mini-Meet
$ grep 'cc\]' results/eval_rate1m_adapt1_p2__botA.log | cut -c1-90 | head -12
16:48:14 [cc] botB reported loss 0.0% rtt -1 ms -> encoder level 2 (500 kbps 480x360)
16:48:15 [cc] botB reported loss 0.0% rtt 0 ms -> encoder level 1 (800 kbps 640x480)
16:48:16 [cc] botB reported loss 0.0% rtt 0 ms -> encoder level 0 (1200 kbps 640x480)
16:48:25 [cc] botB reported loss 0.0% rtt 997 ms -> encoder level 2 (500 kbps 480x360)
16:48:27 [cc] botB reported loss 0.0% rtt 1390 ms -> encoder level 4 (180 kbps 320x240)
16:48:27 [cc] botB reported loss 0.0% rtt 897 ms -> encoder level 5 (100 kbps 160x120)
16:48:39 [cc] botB reported loss 0.0% rtt 3 ms -> encoder level 4 (180 kbps 320x240)
16:48:42 [cc] botB reported loss 0.0% rtt 6 ms -> encoder level 3 (300 kbps 320x240)
16:48:45 [cc] botB reported loss 0.0% rtt 15 ms -> encoder level 2 (500 kbps 480x360)
16:48:48 [cc] botB reported loss 0.0% rtt 38 ms -> encoder level 1 (800 kbps 640x480)
16:48:52 [cc] botB reported loss 0.0% rtt 576 ms -> encoder level 3 (300 kbps 320x240)
16:48:54 [cc] botB reported loss 0.0% rtt 934 ms -> encoder level 5 (100 kbps 160x120)
```

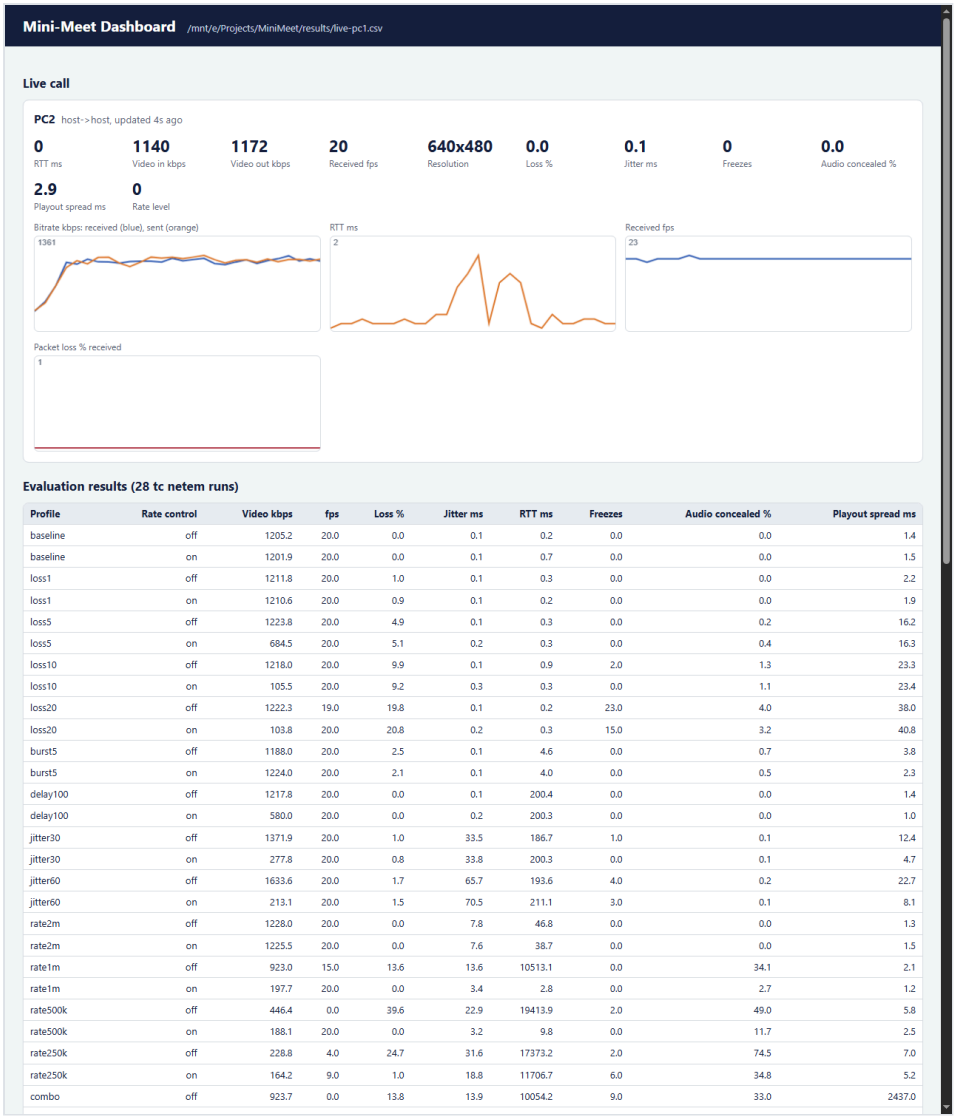
Rate controller log during the 1 Mbit/s run: it backs off when the round-trip time rises, then climbs again.



1 Mbit/s cap applied at 9 s. Fixed rate (blue): the round-trip time climbs past 10 s and loss reaches 13.6 %. With rate control: the call stays at 20 fps.



One call through changing bandwidth: clean, 2 Mbit/s, 1 Mbit/s, 500 kbit/s, 1 Mbit/s, clean.



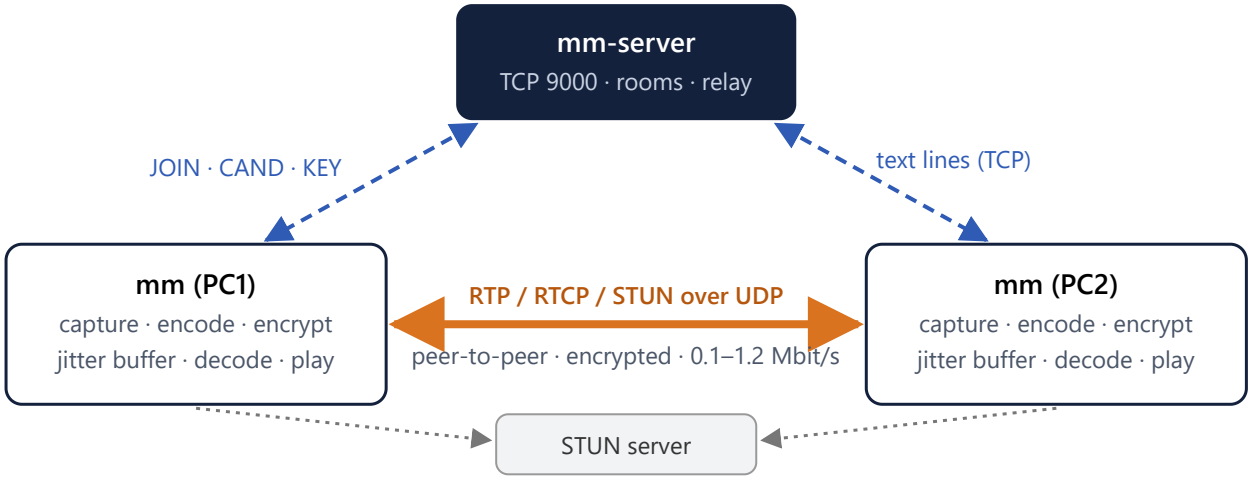
Evaluation results (28 tc netem runs)

Profile	Rate control	Video kbps	fps	Loss %	Jitter ms	RTT ms	Freezes	Audio concealed %	Playout spread ms
baseline	off	1205.2	20.0	0.0	0.1	0.2	0.0	0.0	1.4
baseline	on	1201.9	20.0	0.0	0.1	0.7	0.0	0.0	1.5
loss1	off	1211.8	20.0	1.0	0.1	0.3	0.0	0.0	2.2
loss1	on	1210.6	20.0	0.9	0.1	0.2	0.0	0.0	1.9
loss5	off	1223.8	20.0	4.9	0.1	0.3	0.0	0.2	16.2
loss5	on	684.5	20.0	5.1	0.2	0.3	0.0	0.4	16.3
loss10	off	1218.0	20.0	9.9	0.1	0.9	2.0	1.3	23.3
loss10	on	105.5	20.0	9.2	0.3	0.3	0.0	1.1	23.4
loss20	off	1222.3	19.0	19.8	0.1	0.2	23.0	4.0	38.0
loss20	on	103.8	20.0	20.8	0.2	0.3	15.0	3.2	40.8
burst5	off	1188.0	20.0	2.5	0.1	4.6	0.0	0.7	3.8
burst5	on	1224.0	20.0	2.1	0.1	4.0	0.0	0.5	2.3
delay100	off	1217.8	20.0	0.0	0.1	200.4	0.0	0.0	1.4
delay100	on	580.0	20.0	0.0	0.2	200.3	0.0	0.0	1.0
jitter30	off	1371.9	20.0	1.0	33.5	186.7	1.0	0.1	12.4
jitter30	on	277.8	20.0	0.8	33.8	200.3	0.0	0.1	4.7
jitter60	off	1633.6	20.0	1.7	65.7	193.6	4.0	0.2	22.7
jitter60	on	213.1	20.0	1.5	70.5	211.1	3.0	0.1	8.1
rate2m	off	1228.0	20.0	0.0	7.8	46.8	0.0	0.0	1.3
rate2m	on	1225.5	20.0	0.0	7.6	38.7	0.0	0.0	1.5
rate1m	off	923.0	15.0	13.6	13.6	10513.1	0.0	34.1	2.1
rate1m	on	197.7	20.0	0.0	3.4	2.8	0.0	2.7	1.2
rate500k	off	446.4	0.0	39.6	22.9	19413.9	2.0	49.0	5.8
rate500k	on	188.1	20.0	0.0	3.2	9.8	0.0	11.7	2.5
rate250k	off	228.8	4.0	24.7	31.6	17373.2	2.0	74.5	7.0
rate250k	on	164.2	9.0	1.0	18.8	11706.7	6.0	34.8	5.2
combo	off	923.7	0.0	13.8	13.9	10054.2	9.0	33.0	2437.0

Dashboard (analysis/dashboard.py): live statistics of a running call, above the evaluation results.

Outcome 5: Architecture document covering media path vs. signalling path Done

The full architecture document is `docs/ARCHITECTURE.md` in the repository. In short, signalling goes through the server over TCP, and media goes directly between peers over UDP.



Signalling path (dashed, TCP) and media path (solid, UDP).

	Signalling path	Media path
Purpose	Find peers, exchange network candidates and public keys	Carry audio and video
Endpoints	Client to `mm-server`	Client to client (full mesh)
Transport	TCP, one text line per message	UDP: RTP, RTCP and STUN on one port
Volume	A few hundred bytes per call setup	0.1–1.2 Mbit/s per stream
If the server stops	New joins fail	Running calls continue

Code

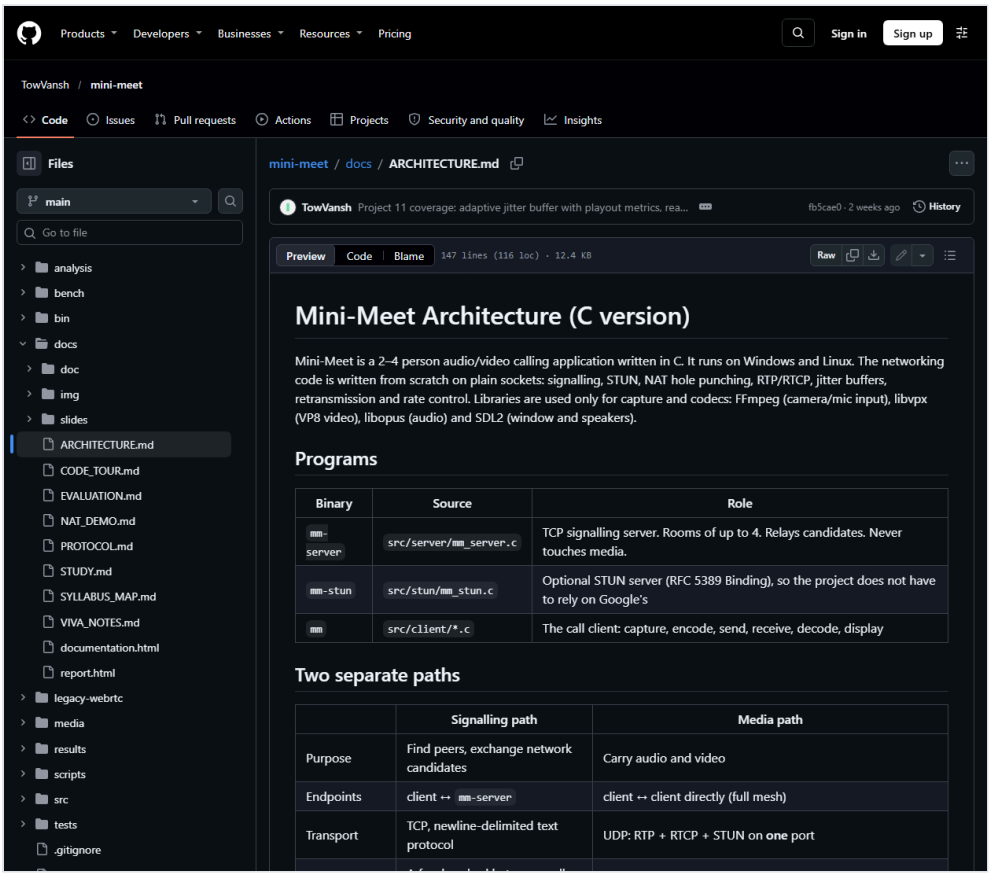
Where the two paths meet in the client: one `select()` loop watches the TCP signalling socket and the UDP media socket, and each UDP packet is sorted into STUN, RTCP or RTP by its first bytes.

`src/client/netloop.c`

```
static void on_udp(uint8_t *buf, size_t len, const endpoint_t *from, uint64_t now_us) {
    peer_t *p;
    rtp_hdr_t h;
    int off;
    size_t wire_len = len;
    /* Demultiplex by first byte (RFC 7983): 0-3 STUN, 128-191 RTP/RTCP. */
    if (stun_is_stun(buf, len)) {
        ice_on_stun(buf, len, from);
        return;
    }
    p = peer_by_addr(from);
    if (!p) return;
    p->last_rx_ms = now_us / 1000;
    if (rtp_is_rtcp(buf, len)) {
        if (g.encrypt && (!p->crypto.ready || !(len = mmc_open_rtcp(&p->crypto, buf, len)))) {
            p->bad_auth++;
            return;
        }
        on_rtcp(p, buf, len, now_us);
        return;
    }
    /* Decrypt and verify before anything else trusts the packet. */
    if (g.encrypt && (!p->crypto.ready || !(len = mmc_open_rtp(&p->crypto, buf, len)))) {
        p->bad_auth++;
```

```
        return;
    }
    off = rtp_parse_header(buf, len, &h);
    if (off < 0) return;
    /* bitrate counts bytes as they were on the wire (including the auth tag) */
    if (h.pt == RTP_PT_VP8 || h.pt == RTP_PT_VP8_RTX) {
        p->rx_bytes_v += wire_len;
        rx_video(p, &h, buf + off, len - (size_t)off, h.pt == RTP_PT_VP8_RTX, now_us);
    } else if (h.pt == RTP_PT_OPUS) {
        p->rx_bytes_a += wire_len;
        rx_audio(p, &h, buf + off, len - (size_t)off, now_us);
    }
}
```

Implementation



The architecture document, docs/ARCHITECTURE.md, in the repository.

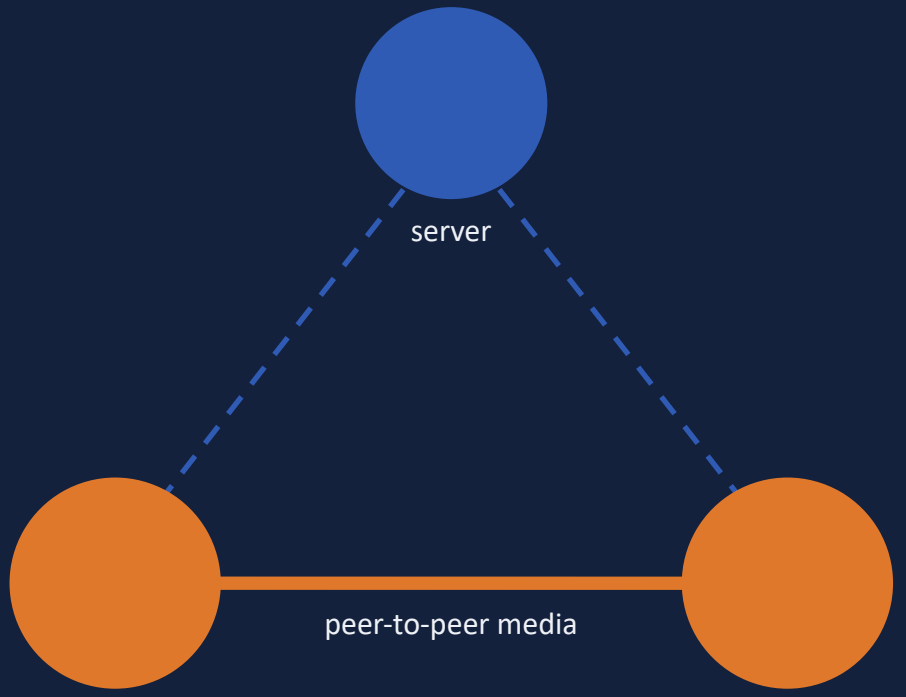


Unit tests covering the protocol code on both paths.

COMPUTER NETWORKS · PROJECT 16 · VIDEO CONFERENCING APPLICATION

Mini-Meet

Real-time video calls written in C: our own signalling, NAT traversal, RTP media engine, jitter buffer, rate control and end-to-end encryption



Suvansh Choudhary 25BAI0252
Priyam Sharma 25BAI0159
Avika Agarwal 25BBS0057

github.com/TowVansh/mini-meet

Expected outcomes: all met

Expected outcome → evidence

Multi-party call: 2–4 people; 12/12 streams verified

Signalling: mm-server, docs/PROTOCOL.md

NAT traversal: STUN, srflx→srflx, Wi-Fi ↔ hotspot

Call quality: 28 tc netem runs, graphs, dashboard

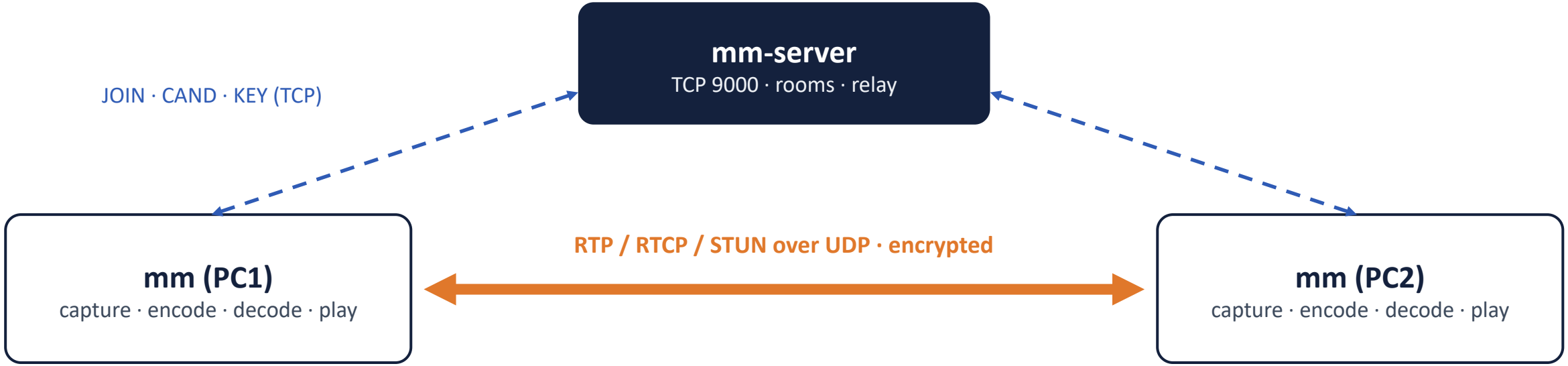
Architecture: media vs signalling, docs/ARCHITECTURE.md

Milestones

1. **Months 1–2**
   signalling server and two-party media path
2. **Months 2–3**
   NAT traversal and multi-party topology
3. **Months 3–4**
   quality instrumentation and adaptation
4. **Months 5–6**
   impairment evaluation, report and viva

Also built: end-to-end encryption · adaptive jitter buffer · Windows + Linux · 72 unit-test checks

Two paths: signalling via the server, media peer-to-peer



Signalling

TCP, newline-delimited text, a few hundred bytes per call. Every message must arrive in order.

Media

UDP, one port for RTP + RTCP + STUN, 0.1–1.2 Mbit/s. Late data is useless, so we repair selectively.

Robust

If the server dies, running calls continue. The server never sees (or can decrypt) media.

Signalling: a plain-text protocol over TCP

```
→ JOIN demo pc3
← JOINED c1 demo a1:pc1
→ CAND a1 srflx 49.36.1.2 50000
→ KEY a1 3f9a…c41d
← CAND a1 host 10.0.0.5 41000
← KEY a1 77be…09aa
→ PING
← PONG
```

1. **Newline framing**

   TCP is a byte stream; lines rebuild message boundaries.

2. **One select() loop**

   Single thread, no locks, same code on Winsock and POSIX.

3. **Validation**

   512-byte lines, name rules, relay only inside a room, max 4.

4. **Liveness**

   PING every 5 s. Clients trust live media over a server timeout.

NAT traversal: STUN plus UDP hole punching

1

Gather

host = LAN IP; srflx = public IP:port from the STUN reply (XOR-MAPPED-ADDRESS)

2

Exchange

candidates sent as CAND lines through the signalling server

3

Punch

both send STUN checks at once; each outgoing packet opens its own NAT

4

Select

first pair that answers carries the media; keepalive every 2.5 s

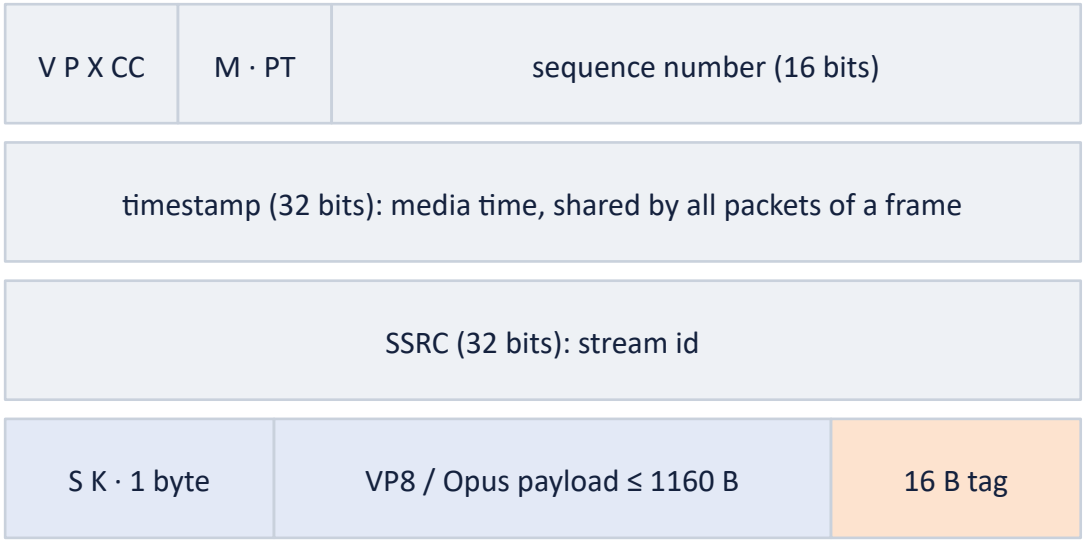
Live test: college Wi-Fi ↔ laptop on a mobile hotspot

Both behind NAT, different ISPs. Connected srflx→srflx (public mapping to public mapping) in 0.1–0.75 s, three times, ~9 minutes of calls. Nothing relayed.

When it fails

Symmetric NAT gives a new port per destination, so STUN's answer is wrong for the peer. Detected after 15 s; would need a TURN relay.

Our framing layer: RTP-style packets on one UDP port



Header stays readable (needed for demultiplexing and statistics) but is authenticated; payload is encrypted.

PT	Payload	Rule
96	VP8, 90 kHz	S = first packet, M = last packet of a frame; split at 1160 B (under the MTU)
97	VP8 resend	same bytes and sequence number; kept out of loss statistics
111	Opus, 48 kHz	one 20 ms frame per packet; timestamp +960 each

One port, demultiplexed by the first byte

- 0–3 + magic cookie → STUN
- 128–191, second byte 200–206 → RTCP (SR, NACK, PLI)
- 128–191 otherwise → RTP

Jitter buffer: play by timestamp, not by arrival

offset = arrival − media time → play media time T at T + p95(last 64 offsets)

About 95 % of packets are there in time; playout follows the sender's pace. The delay grows with jitter and shrinks when the network calms down.

88 % → 0.1 %

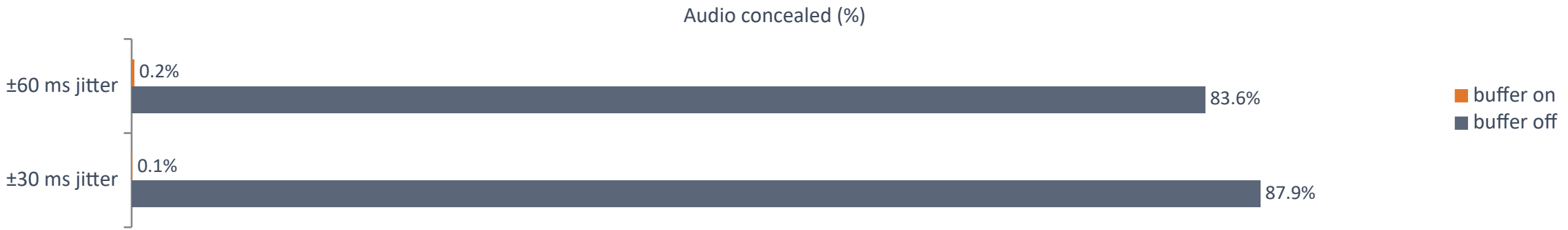
audio frames concealed at ±30 ms jitter (buffer off → on)

25 → 12 ms

spread of video frame intervals (playout smoothness)

+140 ms

the price: extra delay the buffer adds at ±30 ms (≈ 27 ms on a clean link)



Loss repair: resend it, rebuild it, or hide it

1

NACK

Sequence gap → ask for exactly the missing packets (ID + 16-bit mask). Resent every 1.5 × RTT, max 4 tries: selective repeat with a deadline.

2

Resend

Sender keeps the last 1024 video packets and resends them as PT 97, so loss statistics show the network, not the repair.

3

Keyframe

Not repaired in time → skip the frame; later frames depend on it, so request a keyframe (PLI). Seen as a freeze.

4

FEC / PLC

Lost audio is rebuilt from the copy inside the next packet (Opus FEC), otherwise extrapolated (packet loss concealment).

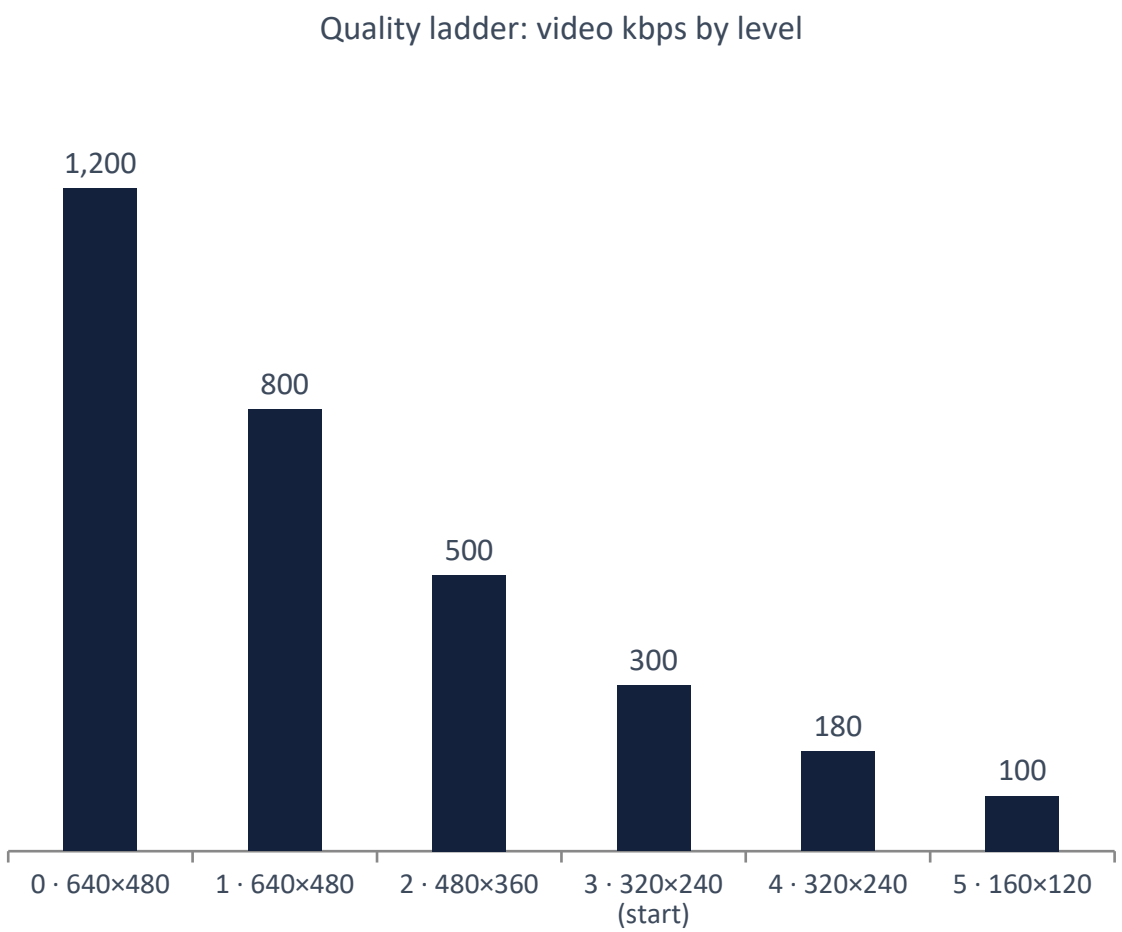
0 freezes

with up to 5 % random loss (≤ 0.4 % of audio concealed)

23 → 15

freezes at 20 % loss, rate control off → on (smaller frames, fewer packets to lose)

Rate control: AIMD driven by receiver reports



−2 Congestion: loss > 8 % or queuing delay > 120 ms → drop two levels (multiplicative decrease)

+1 Clean: loss < 2 % and queuing delay < 40 ms, three reports in a row → climb one level (additive increase)

Δ Queuing delay = smoothed RTT − its 10 s minimum, capped 250 ms above the best ever (delay-based, like BBR / GCC)

S Slow start at 300 kbps; feedback starvation: no report for 2.5 s = stuck in the queue = back off

End-to-end encryption: the server can never see the media

1

Key agreement

Fresh X25519 key pair per run; public keys swapped via the server (KEY line); both sides derive the same 2 session keys

2

Every packet

ChaCha20-Poly1305 AEAD; header readable but authenticated; nonce = SSRC · seq · PT · timestamp

3

Tampering

One flipped bit (even in the sequence number) fails the tag; packet dropped (AUTH-FAIL)

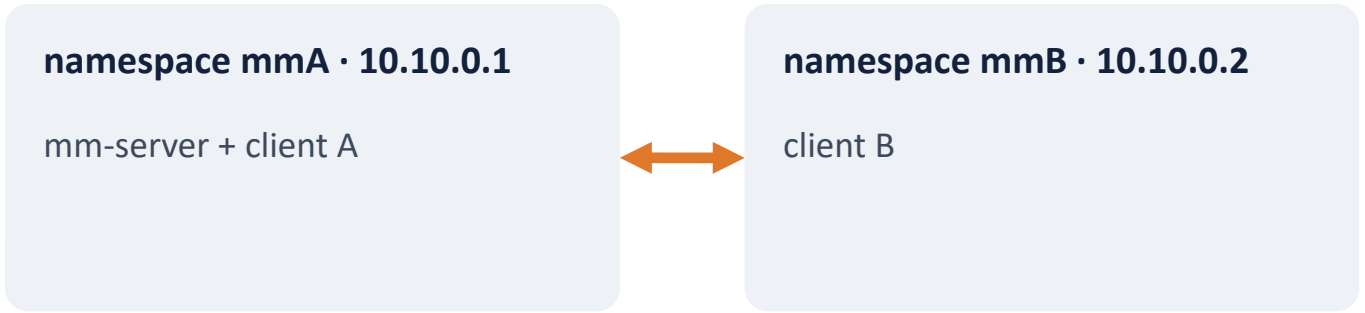
Man-in-the-middle check

A malicious server could swap keys, so both screens show a safety code (hash of both public keys), e.g. 0D66-C339. Same code = no one in between.

Server hardening

512-byte line limit (DoS), strict name/candidate/key checks, relay only within a room, 4-person cap, idle timeout. Not covered: TLS on signalling, media replay window.

Evaluation: 28 runs through emulated bad networks



veth pair, tc netem on both ends: each direction impaired on its own, like a real access link

Profile	netem (each direction)
loss 1 / 5 / 10 / 20	random loss N %
burst5	bursty (Gilbert-Elliott) loss
delay100	100 ms fixed delay
jitter 30 / 60	100 ms ± 30 / 60 ms (reorders)
rate 2m / 1m / 500k / 250k	bandwidth cap, 1000-packet FIFO
combo	80 ± 20 ms, 3 % loss, 1 Mbit/s

- 10 s clean → profile applied mid-call → 45 s recorded
- Rate control off (fixed 1200 kbps) vs on, for every profile
- Per-second CSV: bitrate, fps, loss, jitter, RTT, freezes, concealment, playout spread, buffer delay
- Extra runs: jitter buffer on/off, changing bandwidth in one call, 4-person mesh, real cross-network call

Bandwidth caps: rate control stops bufferbloat

10.5 s → 3 ms

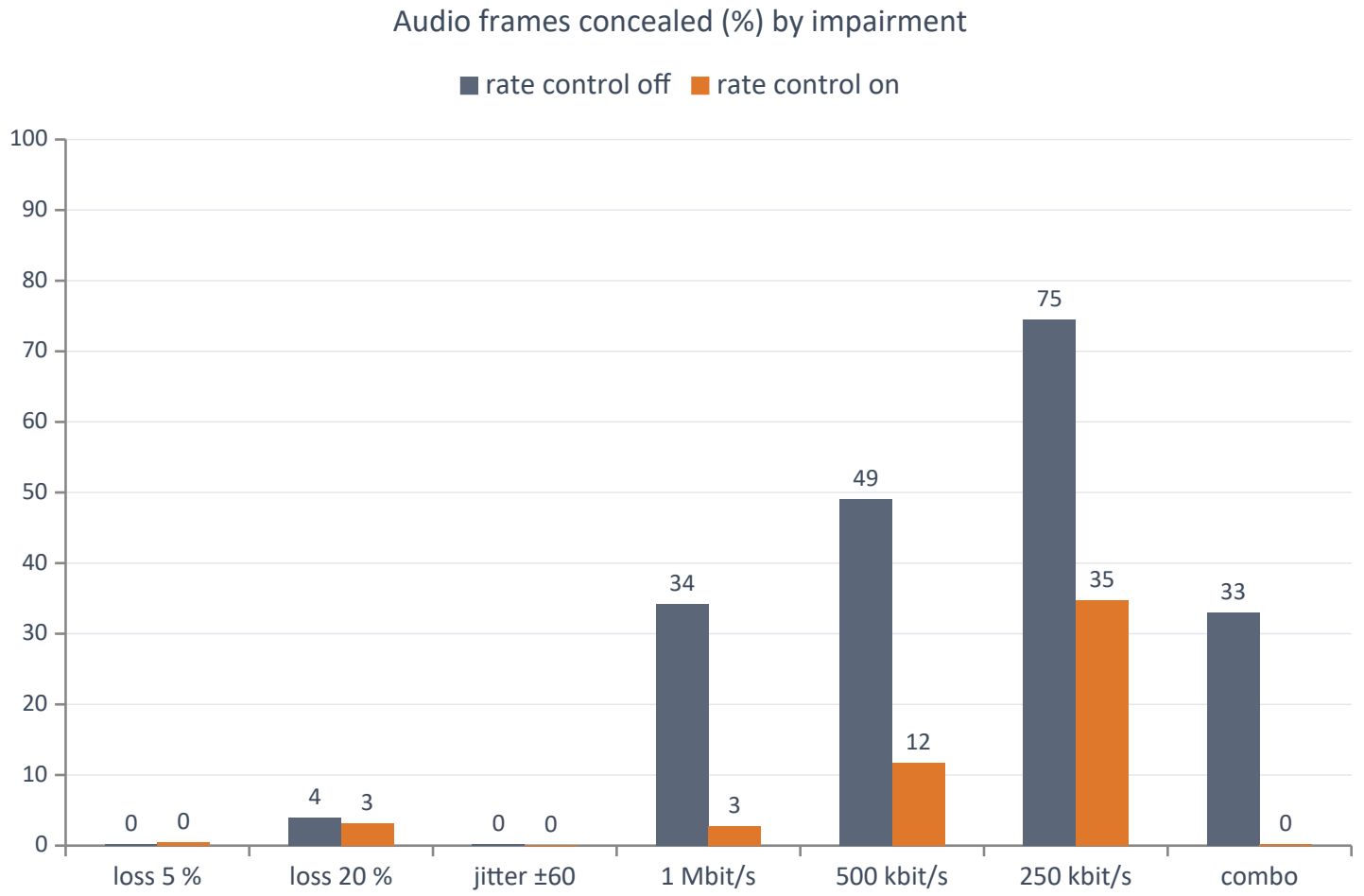
RTT on a 1 Mbit/s link, fixed rate → adaptive

0 → 20 fps

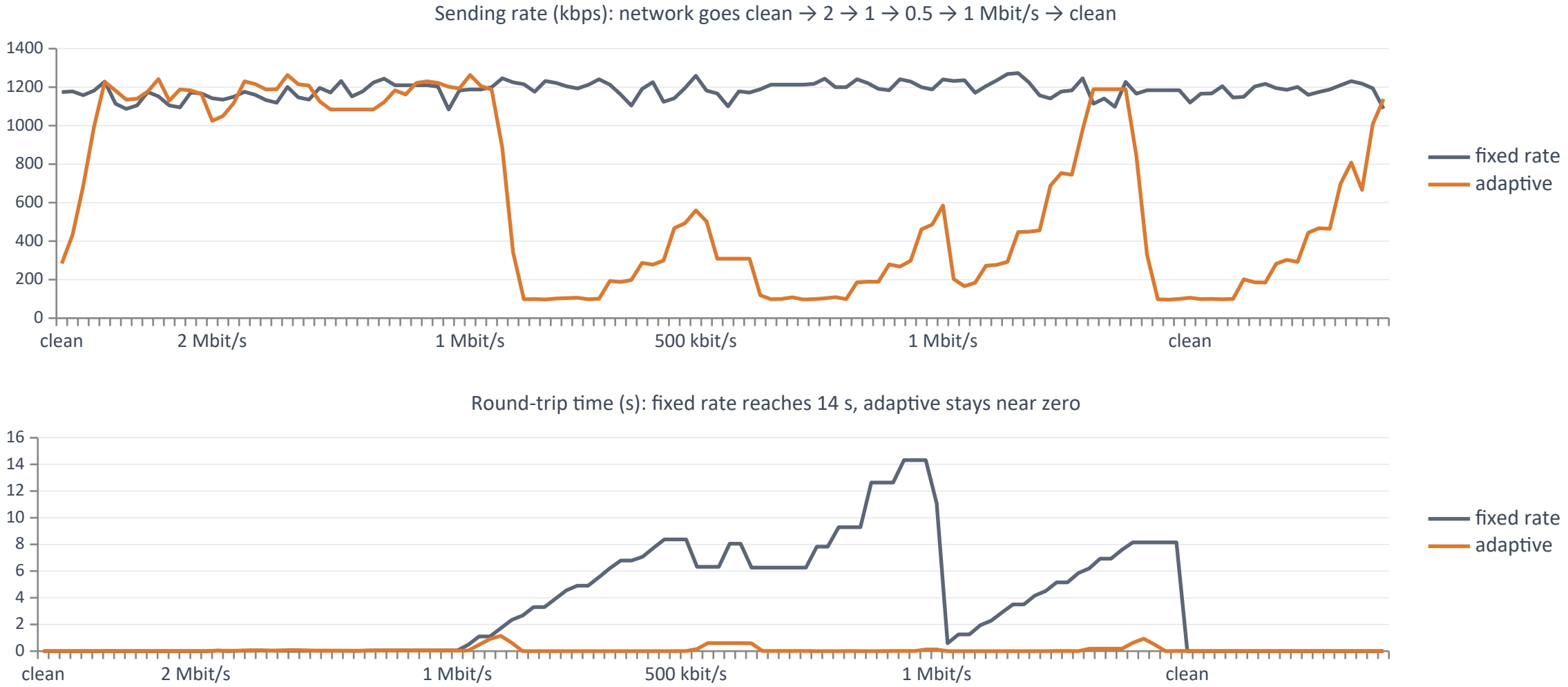
video on a 500 kbit/s link (fixed rate freezes completely)

0 → 20 fps

combined delay + loss + 1 Mbit/s: fixed call stalls, adaptive keeps running



One call through changing bandwidth: the sender follows



Real-world checks beyond the emulator

12 / 12

streams at 640×480, 20 fps in a 4-person mesh (each client uploads 3 copies, ~3.6 Mbit/s)

srflx→srflx

college Wi-Fi ↔ mobile hotspot, median RTT 85 ms; hotspot spikes to 1.15 s were followed by the controller

20 fps

recorded file (H.264 + AAC) streamed in real time, 0 % audio concealed

72 / 72

unit-test checks pass on Windows and Linux: STUN, RTP/RTCP, stats maths, jitter buffer, AIMD, crypto, tamper

Repository and live dashboard

GitHub repository

github.com/TowVansh/mini-meet

- src/: all C code (server, STUN, client)
- tests/: 72 unit-test checks
- bench/, analysis/: netem runs and graphs
- results/: raw data for every run
- docs/: architecture, protocol, evaluation
- bin/: ready-to-run Windows and Linux builds

Dashboard (shown in the live demo)

- Reads the running call's per-second stats file
- Live per peer: RTT, bitrate, fps, loss, jitter, freezes, audio concealment, playout spread
- Live graphs of bitrate, RTT, fps and loss
- Evaluation table for all 28 runs
- Timeline graph of any run

```
python3 analysis/dashboard.py --csv stats.csv
```

Known limits, and what we would do next

Limit	Next step
Random loss and jitter look like congestion	classify loss by delay trend before backing off
250 kbit/s cap: queue already full before feedback returns	emergency pause-video level; AQM (CoDel) in the network
Jitter buffer adds up to ~220 ms under heavy jitter	trade latency vs smoothness per application (voice vs video)
No TURN relay: two symmetric NATs cannot connect	add a TURN fallback
Mesh tops out near 4 people	SFU with simulcast so each receiver gets what its link supports

Takeaways

1. **Both halves of the loop matter**

   jitter buffer at the receiver (88 % → 0.1 % concealed), rate control at the sender (10.5 s → 3 ms RTT)

2. **Everything is our own C**

   sockets, signalling, STUN, framing, repair, rate control and encryption, on Windows and Linux

3. **Measured, not assumed**

   28 impairment runs, jitter-buffer A/B, changing bandwidth, mesh and a real cross-network call

Live demo

`mm-server` · `mm --server 127.0.0.1 --room demo --name PC1` · `mm ... --name PC2 --test`